AI LAB 4

EXPERIMENT 13: Artificial Neural Network Using Backpropagation

AIM: To implement breast tumour classification using an Artificial Neural Network.

QUESTION

A healthcare research system uses tumour measurement data to classify samples into two categories: benign and malignant. An Artificial Neural Network can learn patterns from previously labelled samples by adjusting its weights and biases through backpropagation.

Develop a Python program to implement an Artificial Neural Network using backpropagation.

The program must:

1. Load and preprocess the breast cancer dataset.
2. Construct a neural network with an input layer, one hidden layer, and an output layer.
3. Use the sigmoid activation function.
4. Train the network using forward propagation and backpropagation.
5. Predict the classification of unseen samples.
6. Evaluate the model using classification accuracy and a confusion matrix.

Code:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Load the Breast Cancer Dataset
data = load_breast_cancer()

X = data.data
```

```
y = data.target.reshape(-1, 1)

# Step 2: Preprocess the Dataset
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Initialize Neural Network Parameters
np.random.seed(42)

input_neurons = X_train.shape[1]
hidden_neurons = 10
output_neurons = 1

W1 = np.random.randn(input_neurons, hidden_neurons) * 0.01
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons) * 0.01
b2 = np.zeros((1, output_neurons))

learning_rate = 0.1
epochs = 1000

# Step 4: Define Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

# Derivative of Sigmoid Function
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 5: Train the Neural Network
for epoch in range(epochs):

    # Forward Propagation
    hidden_input = np.dot(X_train, W1) + b1
```

```
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    # Backpropagation
    error = y_train - predicted_output

    d_output = error * sigmoid_derivative(predicted_output)

    error_hidden = np.dot(d_output, W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update Weights and Biases
    W2 += learning_rate * np.dot(hidden_output.T, d_output) / len(X_train)
    b2 += learning_rate * np.mean(d_output, axis=0, keepdims=True)

    W1 += learning_rate * np.dot(X_train.T, d_hidden) / len(X_train)
    b1 += learning_rate * np.mean(d_hidden, axis=0, keepdims=True)

    if (epoch + 1) % 100 == 0:
        print("Epoch:", epoch + 1,
              "Error:", np.mean(np.square(error)))

# Step 6: Predict Unseen Samples
hidden_test = sigmoid(np.dot(X_test, W1) + b1)
output_test = sigmoid(np.dot(hidden_test, W2) + b2)

# Convert probabilities into class labels
y_pred = (output_test >= 0.5).astype(int)

# Step 7: Evaluate the Model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nBreast Cancer Classification Results")
print("------------------------------------")

print("Classification Accuracy:", accuracy * 100, "%")
```

```
print("\nConfusion Matrix:")
print(cm)

print("\nSample Predictions:")
for i in range(10):
    actual = "Benign" if y_test[i][0] == 1 else "Malignant"
    predicted = "Benign" if y_pred[i][0] == 1 else "Malignant"

    print("Actual:", actual, "| Predicted:", predicted)
```

Output:

```
Epoch: 100 Error: 0.228855332357167
Epoch: 200 Error: 0.21613865387018755
Epoch: 300 Error: 0.1965195111819321
Epoch: 400 Error: 0.17217588280380602
Epoch: 500 Error: 0.14742507209776295
Epoch: 600 Error: 0.1254620023476256
Epoch: 700 Error: 0.10746666401668467
Epoch: 800 Error: 0.09323233908986078
Epoch: 900 Error: 0.08204315492851487
Epoch: 1000 Error: 0.07318356206536479

Breast Cancer Classification Results
-----------------------------------
Classification Accuracy: 95.6140350877193 %

Confusion Matrix:
[[38  5]
 [ 0 71]]

Sample Predictions:
Actual: Benign | Predicted: Benign
Actual: Malignant | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
Actual: Benign | Predicted: Benign
Actual: Benign | Predicted: Benign
Actual: Malignant | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
Actual: Benign | Predicted: Benign
Actual: Benign | Predicted: Benign
```

EXPERIMENT 14: Neural Network for Handwritten Digit Recognition

AIM: To implement automated postal code and handwritten digit recognition.

PROBLEM STATEMENT

An automated postal processing system needs to recognize handwritten digits on envelopes to identify PIN codes and route mail to the correct locations. A neural network can learn patterns from handwritten digit images and classify each image into one of ten digit classes (0–9).

Develop an artificial neural network that classifies handwritten digit images into ten categories (0–9). Use the MNIST dataset, preprocess the image data, train the network using backpropagation, and evaluate its classification performance.

Code:

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Load MNIST Dataset
print("Loading MNIST Dataset...")

mnist = fetch_openml('mnist_784', version=1, as_frame=False)

X = mnist.data.astype(np.float32)
y = mnist.target.astype(np.int32)

# Step 2: Preprocess the Data
# Normalize pixel values between 0 and 1
X = X / 255.0

# One-hot encode the target labels
encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y.reshape(-1, 1))

# Use a smaller dataset for faster training
X = X[:10000]
y = y[:10000]
```

```
# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Initialize Neural Network
np.random.seed(42)

input_neurons = 784
hidden_neurons = 128
output_neurons = 10

W1 = np.random.randn(input_neurons, hidden_neurons) * 0.01
b1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons) * 0.01
b2 = np.zeros((1, output_neurons))

learning_rate = 0.1
epochs = 30
batch_size = 100

# Step 4: Define Activation Functions
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_derivative(x):
    return x * (1 - x)

def softmax(x):
    x = x - np.max(x, axis=1, keepdims=True)
    exp_values = np.exp(x)
    return exp_values / np.sum(exp_values, axis=1, keepdims=True)

# Step 5: Train Neural Network Using Backpropagation
print("\nTraining Neural Network...")

for epoch in range(epochs):

    # Shuffle training data
```

```
indices = np.random.permutation(len(X_train))
X_train = X_train[indices]
y_train = y_train[indices]

for start in range(0, len(X_train), batch_size):

    end = start + batch_size

    X_batch = X_train[start:end]
    y_batch = y_train[start:end]

    # Forward Propagation
    hidden_input = np.dot(X_batch, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = softmax(final_input)

    # Backpropagation
    error_output = predicted_output - y_batch

    dW2 = np.dot(hidden_output.T, error_output) / len(X_batch)
    db2 = np.mean(error_output, axis=0, keepdims=True)

    error_hidden = np.dot(error_output, W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    dW1 = np.dot(X_batch.T, d_hidden) / len(X_batch)
    db1 = np.mean(d_hidden, axis=0, keepdims=True)

    # Update Weights and Biases
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

# Calculate training accuracy
hidden = sigmoid(np.dot(X_train, W1) + b1)
output = softmax(np.dot(hidden, W2) + b2)
```

```
    train_pred = np.argmax(output, axis=1)
    train_actual = np.argmax(y_train, axis=1)

    accuracy = accuracy_score(train_actual, train_pred)

    print("Epoch:", epoch + 1,
          "| Training Accuracy:", round(accuracy * 100, 2), "%")

# Step 6: Test the Neural Network
hidden_test = sigmoid(np.dot(X_test, W1) + b1)
output_test = softmax(np.dot(hidden_test, W2) + b2)

y_pred = np.argmax(output_test, axis=1)
y_actual = np.argmax(y_test, axis=1)

# Step 7: Evaluate Model
accuracy = accuracy_score(y_actual, y_pred)

print("\nHandwritten Digit Recognition Results")
print("-------------------------------------")

print("Classification Accuracy:", round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_actual, y_pred))

# Step 8: Display Sample Predictions
print("\nSample Predictions:")

for i in range(10):
    print("Actual Digit:", y_actual[i],
          "| Predicted Digit:", y_pred[i])
```

Output:

```
Loading MNIST Dataset...

Training Neural Network...
Epoch: 1 | Training Accuracy: 21.21 %
Epoch: 2 | Training Accuracy: 34.92 %
Epoch: 3 | Training Accuracy: 55.97 %
Epoch: 4 | Training Accuracy: 60.34 %
```

```
Epoch: 5 | Training Accuracy: 74.46 %
Epoch: 6 | Training Accuracy: 76.34 %
Epoch: 7 | Training Accuracy: 82.39 %
Epoch: 8 | Training Accuracy: 83.73 %
Epoch: 9 | Training Accuracy: 85.51 %
Epoch: 10 | Training Accuracy: 86.16 %
Epoch: 11 | Training Accuracy: 87.19 %
Epoch: 12 | Training Accuracy: 88.0 %
Epoch: 13 | Training Accuracy: 88.76 %
Epoch: 14 | Training Accuracy: 88.84 %
Epoch: 15 | Training Accuracy: 89.41 %
Epoch: 16 | Training Accuracy: 89.53 %
Epoch: 17 | Training Accuracy: 89.83 %
Epoch: 18 | Training Accuracy: 90.06 %
Epoch: 19 | Training Accuracy: 90.49 %
Epoch: 20 | Training Accuracy: 90.44 %
Epoch: 21 | Training Accuracy: 90.6 %
Epoch: 22 | Training Accuracy: 90.92 %
Epoch: 23 | Training Accuracy: 91.03 %
Epoch: 24 | Training Accuracy: 91.2 %
Epoch: 25 | Training Accuracy: 91.5 %
Epoch: 26 | Training Accuracy: 91.31 %
Epoch: 27 | Training Accuracy: 91.65 %
Epoch: 28 | Training Accuracy: 91.64 %
Epoch: 29 | Training Accuracy: 91.94 %
Epoch: 30 | Training Accuracy: 92.11 %

Handwritten Digit Recognition Results
-------------------------------------
Classification Accuracy: 91.65 %

Confusion Matrix:
[[206   0   0   0   0   0   0   0   1   0]
 [  0 208   2   0   0   1   0   0   5   0]
 [  2   1 186   4   3   2   2   0   3   1]
 [  0   2   8 162   0   9   0   4   5   2]
 [  0   1   4   0 199   0   1   0   2   4]
 [  4   3   0  14   0 146   4   0   4   1]
 [  3   0   5   0   2   5 203   0   2   0]
 [  0   1   2   2   2   2   0 203   0   4]
 [  0   3   3   4   1   3   0   0 151   1]
 [  2   0   0   4   3   1   0  12   1 169]]

Sample Predictions:
Actual Digit: 2 | Predicted Digit: 2
Actual Digit: 8 | Predicted Digit: 8
Actual Digit: 6 | Predicted Digit: 6
```

```
Actual Digit: 3 | Predicted Digit: 3
Actual Digit: 8 | Predicted Digit: 8
Actual Digit: 3 | Predicted Digit: 3
Actual Digit: 4 | Predicted Digit: 4
Actual Digit: 0 | Predicted Digit: 0
Actual Digit: 5 | Predicted Digit: 5
Actual Digit: 8 | Predicted Digit: 8
```

EXPERIMENT 15 - Perceptron for Breast Cancer Classification

AIM: To implement breast tumour classification using medical diagnostic measurements.

QUESTION:

Develop a binary classification model using the Breast Cancer Wisconsin Diagnostic dataset. Train a perceptron using tumour measurements and classify samples into benign and malignant categories. Evaluate the model using accuracy and a confusion matrix.

Code:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Load Breast Cancer Dataset
data = load_breast_cancer()

X = data.data
y = data.target

# Step 2: Preprocess the Dataset
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Step 3: Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 4: Create Perceptron Model
```

```
model = Perceptron(
    max_iter=1000,
    eta0=0.01,
    random_state=42
)

# Step 5: Train the Perceptron
model.fit(X_train, y_train)

# Step 6: Predict Test Data
y_pred = model.predict(X_test)

# Step 7: Evaluate the Model
accuracy = accuracy_score(y_test, y_pred)

print("Breast Cancer Classification Results")
print("-------------------------------------")

print("Classification Accuracy:", round(accuracy * 100, 2), "%")

# Step 8: Display Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 9: Display Sample Predictions
print("\nSample Predictions:")

for i in range(10):
    actual = "Benign" if y_test[i] == 1 else "Malignant"
    predicted = "Benign" if y_pred[i] == 1 else "Malignant"

    print("Actual:", actual, "| Predicted:", predicted)
```

Output:

```
Breast Cancer Classification Results
-------------------------------------
Classification Accuracy: 94.74 %

Confusion Matrix:
[[41  1]
```

```
 [ 5 67]]

Sample Predictions:
Actual: Malignant | Predicted: Malignant
Actual: Benign | Predicted: Benign
Actual: Malignant | Predicted: Malignant
Actual: Benign | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
Actual: Benign | Predicted: Benign
Actual: Benign | Predicted: Benign
Actual: Malignant | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
Actual: Malignant | Predicted: Malignant
```

EXPERIMENT 16: Naïve Bayes Classification for Wine Classification

AIM: To implement wine classification using chemical composition analysis.

QUESTION:

In the food and beverage industry, laboratory measurements can help identify wine categories based on their chemical properties. A machine learning model can analyse features such as alcohol content, acidity, magnesium and flavonoids to classify wine samples into their respective categories.

Develop a classification system using the Wine Recognition dataset. Train a Gaussian Naïve Bayes model using the chemical measurements of wine samples and predict their respective categories. Evaluate the model using classification accuracy and a confusion matrix.

Code:

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Step 1: Load Wine Recognition Dataset
data = load_wine()

X = data.data
y = data.target
```

```
# Step 2: Display Dataset Information
print("Wine Classification Dataset")
print("Number of Samples:", X.shape[0])
print("Number of Features:", X.shape[1])
print("Wine Classes:", data.target_names)

# Step 3: Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 4: Create Gaussian Naive Bayes Model
model = GaussianNB()

# Step 5: Train the Model
model.fit(X_train, y_train)

# Step 6: Predict Wine Categories
y_pred = model.predict(X_test)

# Step 7: Evaluate Model Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nWine Classification Results")
print("----------------------------")

print("Classification Accuracy:", round(accuracy * 100, 2), "%")

# Step 8: Display Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Step 9: Display Sample Predictions
print("\nSample Predictions:")

for i in range(10):
    actual = data.target_names[y_test[i]]
```

```
    predicted = data.target_names[y_pred[i]]

    print("Actual:", actual, "| Predicted:", predicted)
```

Output:

```
Wine Classification Dataset
Number of Samples: 178
Number of Features: 13
Wine Classes: ['class_0' 'class_1' 'class_2']

Wine Classification Results
---------------------------
Classification Accuracy: 97.22 %

Confusion Matrix:
[[12  0  0]
 [ 1 13  0]
 [ 0  0 10]]

Sample Predictions:
Actual: class_0 | Predicted: class_0
Actual: class_2 | Predicted: class_2
Actual: class_0 | Predicted: class_0
Actual: class_1 | Predicted: class_1
Actual: class_1 | Predicted: class_1
Actual: class_0 | Predicted: class_0
Actual: class_0 | Predicted: class_0
Actual: class_1 | Predicted: class_0
Actual: class_1 | Predicted: class_1
Actual: class_2 | Predicted: class_2
```